King Fahd University of Petroleum & Minerals
College of Computing and Mathematics
Department of Computer Science

ICS 433 – Operating Systems

Memory Allocation Simulator

Phase 1: Detailed Requirements & Solution Design

Group Members

Student ID	Student Name	Role
202280020	Ali Alsarhayd	Team Leader
201948610	Abdulkarim Althani	Team member
202039820	Mohammed Alrasasi	Team member
202263680	Hassan Alhassan	Team member

Table of Contents

Section 1: Detailed Project Requirements

1.1 High-Level Summary

The Memory Allocation Simulator is an interactive, educational desktop application designed to demonstrate how operating systems manage physical memory by allocating space to processes. The simulator allows a user to define the total size of available memory, create processes with specific memory demands, and then observe how different memory allocation algorithms place those processes into memory. The system visually represents the memory map at each step, clearly showing occupied regions, free holes, and relevant metadata such as fragmentation and wasted space.

This project serves as both a learning tool and a demonstration of core operating system concepts including contiguous memory allocation, internal/external fragmentation, and the trade-offs between allocation strategies. It targets students and educators in operating systems courses.

1.2 Functional Requirements

FR-01	**Memory Configuration** The user shall be able to define the total size of the memory (in MB or KB) at the start of each simulation session. The memory size must be a positive integer greater than zero.
FR-02	**Process Management** The user shall be able to add a process by specifying its name/ID and the amount of memory it requires. The user shall also be able to remove an existing process and release its allocated memory back to the free pool.
FR-03	**Algorithm Selection** The system shall support the following memory allocation algorithms, selectable by the user: • First Fit: Allocate the first free hole that is large enough. • Best Fit: Allocate the smallest free hole that is large enough. • Worst Fit: Allocate the largest free hole available. • Next Fit: Like First Fit but begins searching from where the last allocation ended.
FR-04	**Memory Allocation** Upon adding a process, the system shall apply the selected algorithm to find a suitable free hole. If a suitable hole is found, the process is placed in that memory region. If no hole is large enough, the system shall notify the user with an appropriate message.
FR-05	**Memory Visualization** The system shall display a graphical or ASCII representation of the memory map, showing: allocated blocks (labelled with process name and size), free holes (labelled with their size), and the starting and ending address of each block.
FR-06	**Statistics Display** The system shall compute and display: total memory used, total free memory, number of free holes, degree of external fragmentation, and utilization percentage.

FR-07	**Memory Compaction** The user shall be able to trigger a compaction operation that merges all free holes into one contiguous block at the end of the memory space, simulating OS-level memory compaction.
FR-08	**Reset / New Session** The user shall be able to reset the simulator entirely, clearing all processes and resetting the memory to its initial empty state.
FR-09	**Step-by-Step Mode** The simulator shall optionally support a step-by-step mode where the user can observe how the algorithm scans and selects a hole before confirming the allocation.

1.3 Non-Functional Requirements

- **Usability:** The interface shall be intuitive and require no prior technical expertise. Labels, buttons, and visual cues must be clearly understandable.
- **Performance:** Allocation and visualization updates must complete within 1 second for memory sizes up to 10,000 units and up to 50 concurrent processes.
- **Accuracy:** The simulator must produce allocation results that are mathematically correct and consistent with standard algorithm definitions.
- **Portability:** The application shall run on Windows, macOS, and Linux without modification.
- **Maintainability:** Code shall be modular and well-commented to allow easy extension (e.g., adding new algorithms).
- **Reliability:** The application shall handle invalid inputs gracefully (e.g., process size exceeding total memory) with clear error messages and no crashes.

1.4 User Interface Requirements

- A control panel to set total memory size and select the allocation algorithm.
- An input form to add a new process (name and size).
- A scrollable list of current processes in memory.
- A visual memory map panel that updates dynamically after each operation.
- A statistics panel showing memory utilization metrics.
- Buttons for: Add Process, Remove Process, Compact Memory, Reset, and Toggle Step Mode.

Section 2: Solution Design

2.1 Implementation Platform

Component	Choice	Justification
Programming Language	Python 3.11+	Easy to write, cross-platform, rich ecosystem.
GUI Framework	Tkinter (built-in) or PyQt5	Tkinter requires no extra install; PyQt5 offers richer widgets.
Visualization	Canvas widget / matplotlib	Allows drawing custom memory block diagrams.
Data Layer	In-memory Python lists/objects	No database needed; state is session-based.
Target OS	Windows, macOS, Linux	Python and Tkinter are cross-platform.
Version Control	Git / GitHub	Collaboration and history tracking.

2.2 System Architecture

The simulator follows a three-layer Model-View-Controller (MVC) architectural pattern:

Model (Memory Manager)

– Maintains the memory state as a list of memory blocks (each block is either allocated or free).

– Exposes methods: allocate(process, algorithm), deallocate(process_id), compact(), reset().

– Contains the four algorithm implementations as separate strategy functions.

View (GUI Layer)

– Renders the memory map as a proportional bar chart drawn on a Canvas widget.

– Displays process list, statistics panel, and control widgets.

– Subscribes to model change events and redraws automatically.

Controller (Event Handler)

– Handles user interactions (button clicks, form submissions).

– Validates inputs before forwarding to the model.

– Communicates results back to the view.

2.3 Algorithm Designs

First Fit	Scan the hole list from the beginning. Allocate the first hole whose size >= process size. Split the hole: assign the required portion to the process, leave the remainder as a new smaller hole. *Complexity: O(n) — fast but may create many small fragments near the start of memory.*
Best Fit	Scan the entire hole list and select the smallest hole that is still >= process size. This minimizes wasted space within the chosen hole but requires a full scan each time. *Complexity: O(n) — produces the smallest leftover hole, reducing wasted space per allocation.*
Worst Fit	Select the largest available hole for allocation. The idea is that the leftover fragment is large enough to be useful for future processes. *Complexity: O(n) — may preserve usable fragments but tends to deplete large holes quickly.*
Next Fit	A variant of First Fit that remembers the position of the last allocation. The next search starts from that position, wrapping around to the beginning if needed. *Complexity: O(n) — spreads allocations more uniformly across memory than First Fit.*

2.4 Data Structures

The core data structure is a **list of MemoryBlock objects**, representing memory as a sequence of contiguous segments. Each block stores:

Field	Type	Description
`start_address`	int	Starting byte address of the block
`size`	int	Size of the block in chosen units

`is_free`	bool	True if the block is a free hole, False if allocated
`process_id`	str | None	Identifier of the process occupying this block (None if free)
`process_name`	str | None	Human-readable name of the process

Adjacent free blocks are automatically merged (coalesced) after every deallocation to prevent artificial fragmentation from appearing in the hole list.

2.5 User Interface Design

The GUI is divided into three primary panels arranged horizontally:

Left Panel – Controls

– Total memory size input (text field + unit selector KB/MB)
– Algorithm selector (radio buttons or dropdown)
– Process name and size inputs
– Add Process / Remove Selected / Compact / Reset buttons
– Step Mode toggle checkbox

Center Panel – Memory Map

– A proportional vertical or horizontal bar representing the full memory
– Each segment colored distinctly: blue for allocated blocks, grey for free holes
– Labels inside each segment: process name, size, and address range
– Tooltip on hover showing full block details

Right Panel – Statistics

– Total Memory, Used Memory, Free Memory (with percentage)
– Number of free holes
– Largest free hole size
– External fragmentation percentage
– Process list table with columns: Name, Size, Start Address, End Address

Section 3: Constraints, Assumptions & Contributions

3.1 Constraints and Assumptions

C-01	**Contiguous Allocation Only**: This simulator models contiguous memory allocation only. Segmentation and paging are out of scope for Phase 1.
C-02	**Single-Level Memory**: The simulator models a flat, single-level physical memory space. Virtual memory and multi-level hierarchies are not simulated.
C-03	**No Preemption**: Once a process is allocated memory, it remains in memory until explicitly removed by the user. The simulator does not model preemptive reclamation.
C-04	**Integer Memory Units**: All memory sizes (total memory and process requests) are assumed to be positive integers. Fractional units are not supported.
C-05	**Maximum Memory Size**: The simulator supports memory sizes up to 1,000,000 units to ensure visualization remains legible and performance stays acceptable.
C-06	**Maximum Processes**: A maximum of 100 concurrent processes may be added per session, beyond which the user is notified.
C-07	**No Persistence**: All session data is in-memory and is lost when the application is closed. There is no save/load functionality in Phase 1.
C-08	**Process Size Limit**: No single process may request more memory than the total configured memory size.
A-01	**Uniform Address Space**: The memory address space starts at address 0 and ends at (total_size - 1).
A-02	**Immediate Coalescing**: Adjacent free holes are merged immediately after a process is deallocated, keeping the hole list minimal.
A-03	**Block Granularity**: Allocation is done at 1-unit granularity (no alignment padding), simplifying the model for educational clarity.

3.2 Group Member Contributions

Member	ID	Phase 1 Contribution
Ali Alsarhayd	202280020	25%
Abdulkarim Althani	201948610	25%
Mohammed Alrasasi	202039820	25%
Hassan Alhassan	202263680	25%